

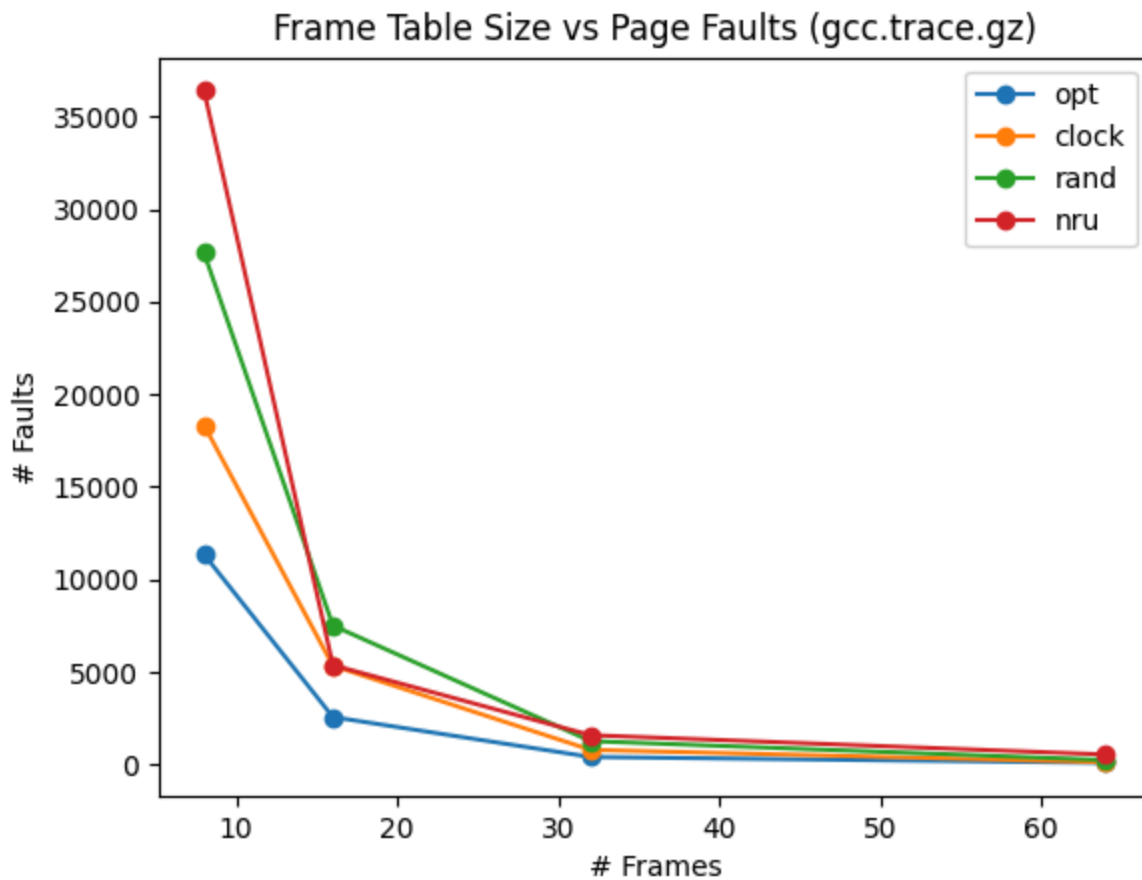
CSC 452 - Homework 5 Write-Up

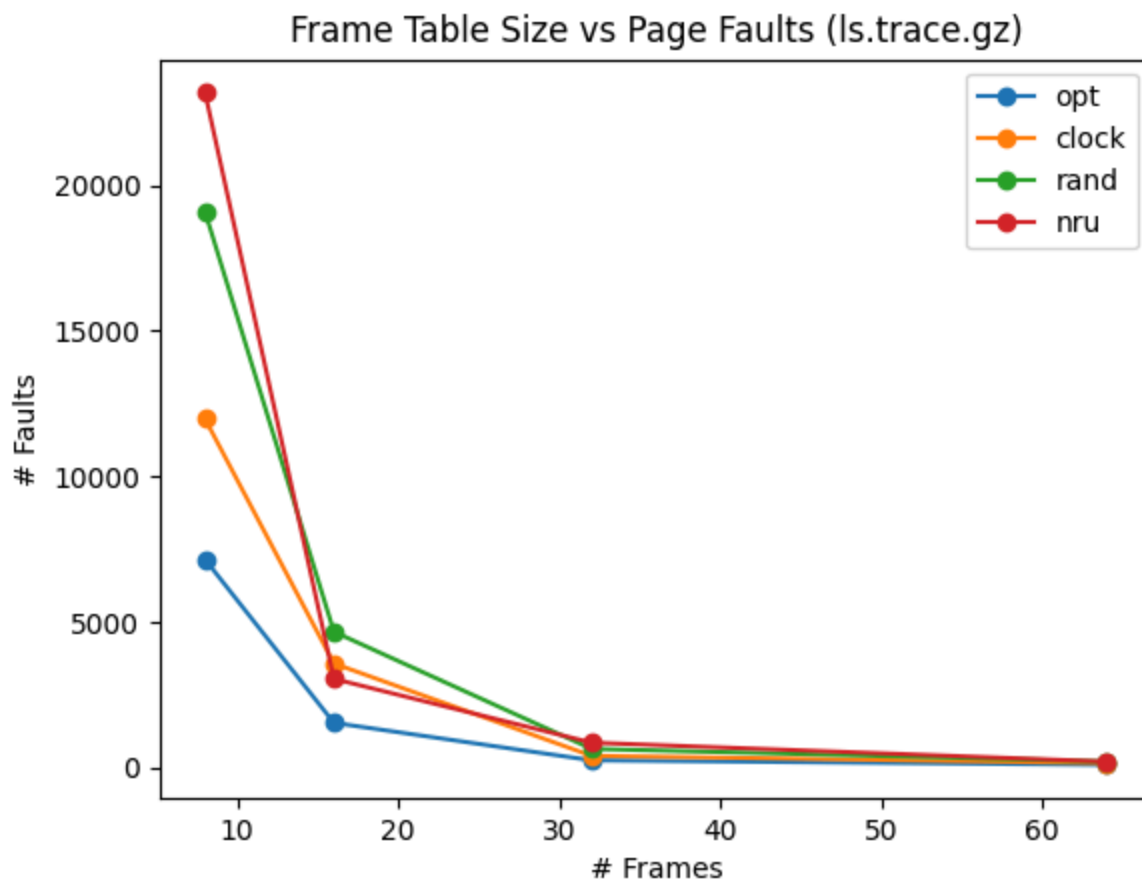
Gabe Venegas

5/1/26

Performance Graphs

As seen below, for both instruction traces used in testing, there exists a negative correlation (potentially negative-exponential) between number of frames in frame pool and the count of page faults occurred during runtime, for all algorithms tested, where the optimal algorithm (opt) seems to safely cut below any other algorithm tested.





Discussion

OPT

The optimal (opt) algorithm acts as a golden baseline from which each algorithm could only strive to match, as it's the only algorithm with the ability to see into the future all times a page is referenced for the process' lifetime. Due to it's ability to evict the page that won't be needed for the longest time, it is able to prevent as many page faults as possible by allowing as many "earlier" pages as possible to have their go before we need to re-admit the evicted page. Although as we see, for smaller frame table sizes, opt does not experience zero page faults. This suggests there still exists the issue of bottlenecking on the given "working set" as we provide less and less frames to load pages into, even in this optimal case.

While opt is physically impossible to implement as a real OS cannot predict all future events in real time, plotting it acts as a good indicator for how much performance we can expect to squeeze out of each "realistic" algorithm at it's best.

Clock

Clock (also known as "second chance") traces the "staleness" of pages, choosing to evict the oldest unreferenced page in the frame table, marking any passed-over referenced pages as

stale during search to be picked on next cycling of the clock hand (if not re-referenced by the time of next eviction selection).

In testing, as shown in the above graphs, proved to be the most consistent "real-world" page replacement algorithm. Across all scenarios, it held the tightest curve much closer to opt than any other algorithm tested. Because it respects temporal locality (recently accessed pages) it could be said that this algorithm could be a best-choice (of those tested) in a real OS as it would also be a very memory-stable algorithm.

NRU

The NRU (not recently used) algorithm operates similar to the clock algorithm, in the sense that it attempts to choose pages based on "staleness". Unlike clock, NRU instead expires page references every fixed interval; where if a page is not re-referenced before the next eviction (in this interval) it becomes an eviction candidate as we denoted it as stale/unused on last interval.

For the refresh interval, a statically-set refresh parameter was selected and used for all cases. A value of 500 was determined via manual trial-and-error using a frame count of 16 as it appeared to provide approximately the minimal page faults for this frame count.

In testing, for the low frame scenarios, NRU racked up thousands of more faults than simply choosing at-random (rand), and still slightly more faults than all other algorithms even when frame count grew larger. As we can see, once we hit 16 frames, NRU actually managed to outperform clock. This suggests that when NRU is properly tuned for the amount of page frames it has to work with, it makes very smart eviction choices. That said, it holds a high sensitivity to the refresh parameter, making it a bit of a gamble for general-purpose use where frame table size may be dynamic or unknown.

Rand

The random (rand) algorithm doesn't care about anything regarding environment or strategic eviction; it just picks a victim and moves on. While this makes it very easy to implement with minimal overhead, the graph shows why it's far from best. It consistently faults more than clock and fails to consider the logic of how programs actually access memory. Its only redeeming quality is that it wasn't as bad as NRU for the frame sizes NRU wasn't tuned for, but it quickly fell behind as more memory became available.

Summary

In a real-world system, clock appeared to be the most suitable of the algorithms tested. It offered the perks of low overhead and good predictability. While NRU has the ability to slightly outperform clock, it only does so at a specific frame count, once refresh parameter is tuned for that frame count and underperforms for all other frame counts. For a general-purpose system

that needs to run both very small and very large programs where each 16 KB miss is a performance hit, clock provides more across-the-board consistency.